

S08_LP2_Equipo02_ApazaMamaniWilsonRonald.pdf

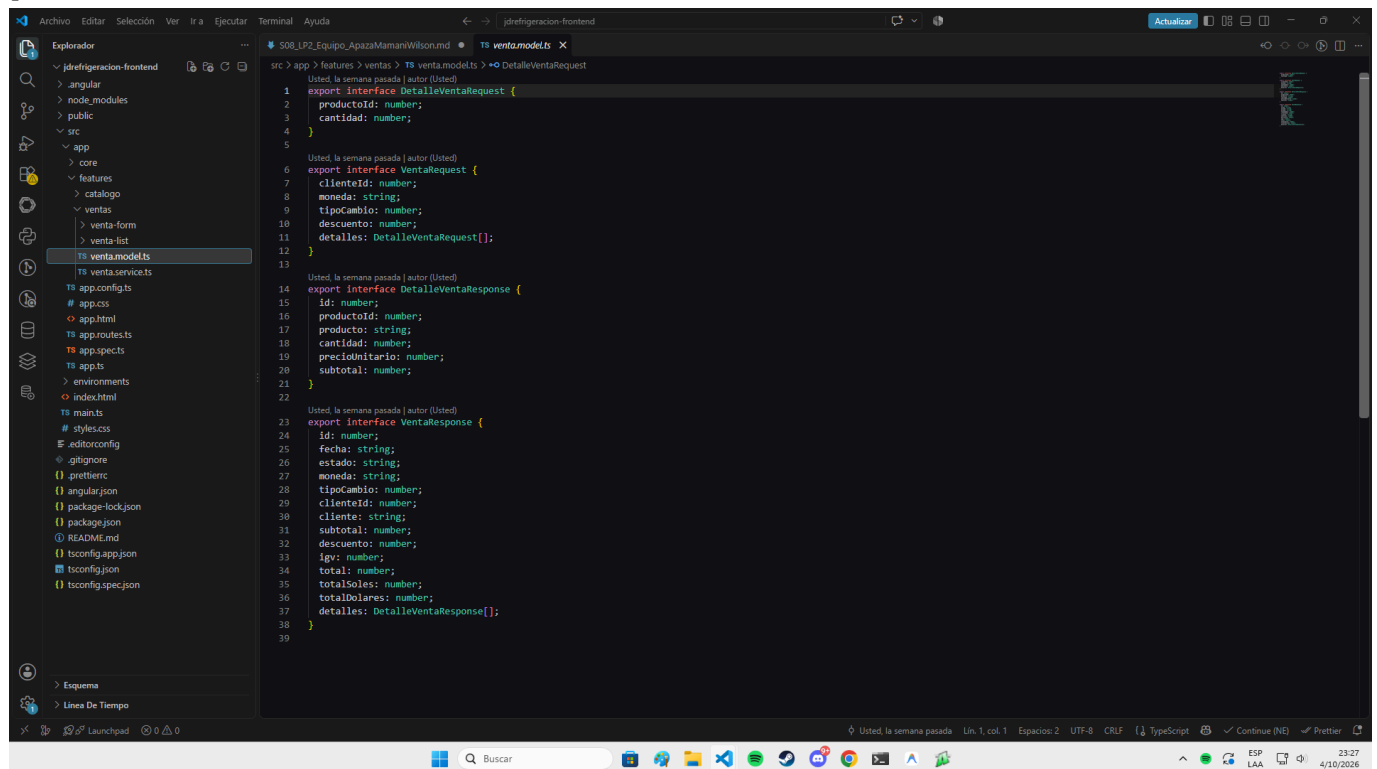
Datos del estudiante

- **Nombre:** Wilson R. Apaza Mamani
- **Equipo:** [02/CHERRY TREE]
- **Sesión:** S08 - CRUD de Tablas Dependientes
- **Rol o aporte realizado:** Desarrollo Full Stack del módulo de Ventas y DetalleVenta.
- **Link de GitHub:** <https://github.com/ApazaMamaniWilsonR/JDRefrigeracion>

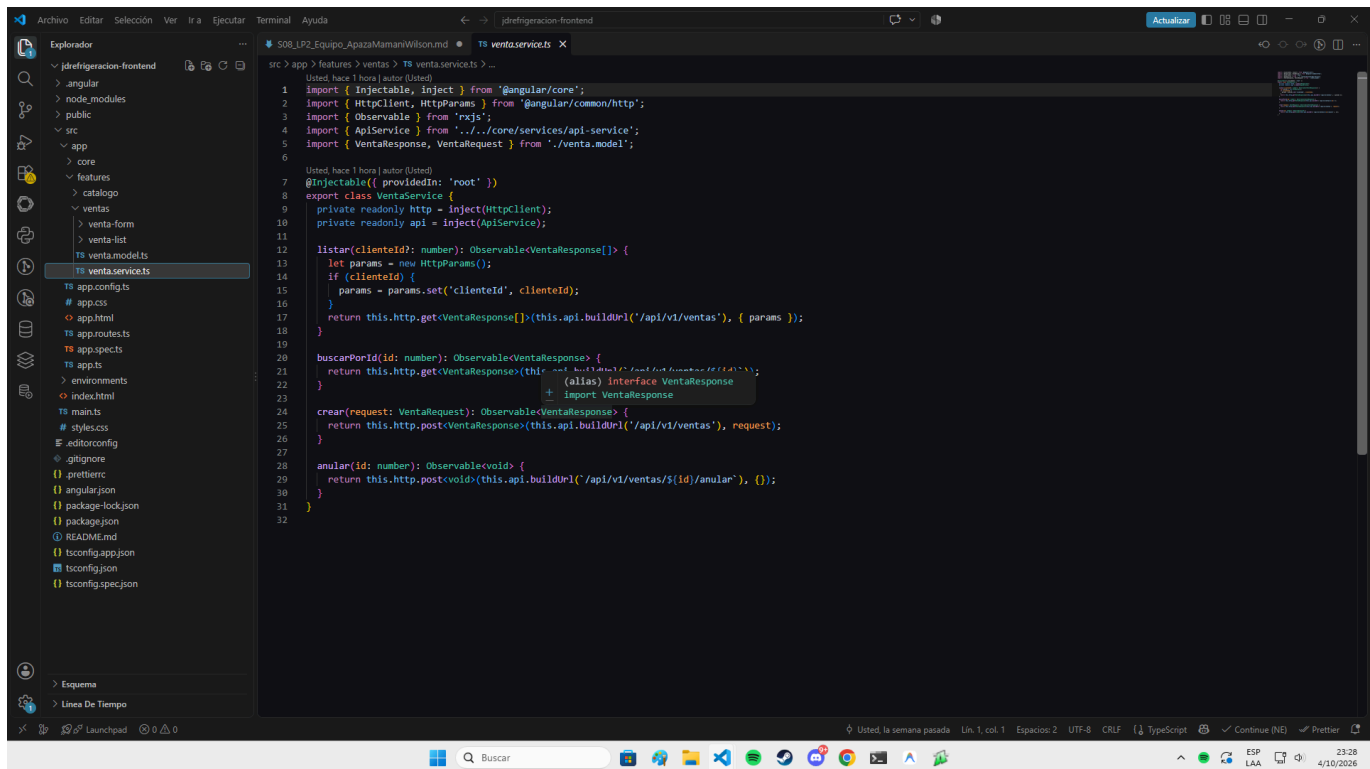
Evidencia técnica

1. Modelos y servicios

[



: Modelos VentaRequest y VentaResponse] [



: VentaService con Llamadas HTTP]

- **Explicación:** Se definieron dos modelos distintos para la tabla dependiente **Venta**: **VentaRequest** para enviar datos al backend (incluyendo solo el **clienteId**), y **VentaResponse** para recibir los datos de la base de datos (incluyendo el nombre del cliente y los subtotales calculados). El servicio **VentaService** maneja las peticiones HTTP exclusivas de esta entidad de forma separada de los componentes. Ningún componente inyecta **HttpClient** directamente.
- **Código de Modelos (venta.model.ts):**

```
export interface VentaRequest {
  clienteId: number;
  moneda: string;
  tipoCambio: number;
  descuento: number;
  detalles: DetalleVentaRequest[];
}

export interface VentaResponse {
  id: number;
  fecha: string;
  estado: string;
  clienteId: number;
  cliente: string;
  total: number;
  detalles: DetalleVentaResponse[];
}
```

2. Lista con información relacionada

JD Refrigeración

Inicio
Categorías
Ventas

Ventas Realizadas

Filtrar por Cliente: Juan Perez [Nueva Venta](#)

ID	Fecha	Cliente	Moneda	Total	Estado	Acciones
1	10/2/26, 11:33 AM	Juan Perez	PEN	PEN1,200.00	REGISTRADA	Ver Detalle Anular

DevTools Network: 1 requests, 665 B transferred, 361 B resources. Request: ventas?clienteId=1, Status: 200, Type: fetch, Initiator: venta-list.ts:30, Size: 0.7 kB, Time: 7 ms.

: Pantalla de Lista de Ventas mostrando el filtro y la tabla] [

JD Refrigeración

Inicio
Categorías
Ventas

Ventas Realizadas

Filtrar por Cliente: Juan Perez [Nueva Venta](#)

ID	Fecha	Cliente	Moneda	Total	Estado	Acciones
1	10/2/26, 11:33 AM	Juan Perez	PEN	PEN1,200.00	REGISTRADA	Ver Detalle Anular

DevTools Network: 1 requests, 665 B transferred. Request: ventas?clienteId=1, Headers: Query string parameters, clienteId: 1.

: Pestaña Network mostrando el parámetro GET]

- **Explicación:** La tabla de lista muestra correctamente la relación con la tabla padre, enseñando el nombre del cliente en lugar de su ID, gracias al modelo `VentaResponse`. Además, se implementó un filtro en la cabecera que hace una petición directa al servidor enviando el parámetro `clienteId`, resolviendo el filtrado en el backend y no en el navegador.
- **Código del Servicio y Filtro (`venta.service.ts` y `venta-list.ts`):**

```
listar(clienteId?: number): Observable<VentaResponse[]> {
  let params = new HttpParams();
```

```

    if (clienteId) params = params.set('clienteId', clienteId);
    return this.http.get<VentaResponse[]>(this.api.buildUrl('/api/v1/ventas'), {
    params });
  }

```

3. Lista desplegable y formulario

[

The screenshot shows a web browser window with the URL 'localhost:4200/ventas/nuevo'. The application is 'JD Refrigeración'. The left sidebar has links for 'Inicio', 'Categorías', and 'Ventas' (which is active). The main content area is titled 'Nueva Venta'. Under 'Datos Principales', there's a 'Cliente' dropdown with a list of clients, and a 'Moneda' dropdown set to 'Soles (PEN)'. Below these is a 'Detalle de Venta' section with a table. The table has columns: 'Producto' (dropdown), 'Cantidad' (input with value 1), 'P. Unitario' (PEN0.00), and 'Subtotal' (PEN0.00). At the bottom of the table is a 'Total Aprox (Inc. IGV):' row with the value PEN0.00. There are buttons for 'Agregar Producto', 'Completar Venta', and 'Cancelar'.

: Formulario de Nueva Venta con el Select abierto]

- Explicación:** El formulario de ventas reutiliza el `ClienteService` para poblar dinámicamente la lista desplegable de la tabla padre (Cliente) y el `ProductoService` para el detalle dinámico. Los selectores utilizan la directiva `[ngValue]` para asegurar que el dato seleccionado se guarde en memoria como un dato numérico estricto y no como texto. (Nota: Por las reglas de negocio del dominio de facturación/ventas, estos documentos no se editan tras su emisión, solo se anulan, por lo que no aplica preselección de edición en este caso particular).
- Código del Select con `ngValue` (venta-form.html):**

```

<select formControlName="clienteId">
  <option [ngValue]="null">-- Seleccione un Cliente --</option>
  @for (cli of clientes(); track cli.id) {
    <option [ngValue]="cli.id">{{ cli.numeroDocumento }} - {{ cli.nombres ||
cli.razonSocial }}</option>
  }
</select>

```

4. Validación de dependencias

: Formulario mostrando el mensaje "Seleccione un cliente válido" o "No hay clientes registrados"] [

: Alerta del navegador que salta cuando el backend responde 404]

- **Explicación:** Las dependencias se validan en dos niveles. En el formulario, se bloquea el guardado y se muestra un mensaje si el usuario intenta enviar datos sin elegir una opción de la lista. En el segundo nivel, se intercepta el error HTTP del backend; si este responde con un error 404 específico (porque el cliente o el producto fue eliminado mientras el formulario estaba abierto), la pantalla lo identifica y recarga las listas automáticamente para restaurar el estado real.
- **Código del manejo de 404 (venta-form.ts):**

```
private manejarErrorGuardado(err: HttpResponse): void {
  const mensaje = err.error?.message ?? '';
  if (err.status === 404 && mensaje.toLowerCase().includes('cliente')) {
    alert('El cliente seleccionado ya no existe. La lista se recargará.');
```

```
    this.form.controls['clienteId'].setValue(null);
    this.clienteService.listar().subscribe(data => this.clientes.set(data));
  }
}
```

Error o hallazgo

Durante el desarrollo de la interfaz del formulario, descubrí que al usar el atributo `[value]` estándar de HTML en las opciones del selector de cliente (`<option [value]="cli.id">`), Angular estaba capturando el ID como una cadena de texto (ej. "3") al momento de construir el `VentaRequest`. Si bien el backend de Java lo lograba convertir automáticamente a `Long`, este comportamiento podía causar problemas en comparaciones estrictas (`===`) en TypeScript.

La solución técnica fue reemplazar el atributo estándar por la directiva de Angular `[ngValue]="cli.id"`. Esto garantizó que el valor asociado al `FormControl` mantuviera estrictamente el tipo numérico.

Reflexión técnica breve

¿Por qué el formulario valida que se haya elegido una categoría (o cliente) si el backend ya rechaza una referencia inexistente — y qué pasaría si solo existiera una de las dos validaciones?

Ambas validaciones tienen responsabilidades distintas. Si solo validara el backend, el usuario tendría una mala experiencia teniendo que esperar al envío del formulario para enterarse del error, y haríamos llamadas a la API innecesarias que cargarían al servidor. Por otro lado, si solo validara el frontend, la base de datos quedaría desprotegida, ya que cualquier atacante (o un script malicioso) podría consumir nuestra API directamente con herramientas externas saltándose la interfaz y enviando datos corruptos o falsos a nuestro sistema.

Anexo: Feedback de la sesión

1. **¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?** El aprendizaje más importante fue entender cómo separar correctamente los modelos de datos (Request y Response) y comprender la importancia de usar `[ngValue]` en lugar de `[value]` para evitar enviar tipos de texto (String) cuando el backend espera números (Long).
2. **¿Qué punto de la clase te resultó más confuso o te dejó con dudas?** La parte más confusa al inicio fue entender cómo manejar los errores 404 de manera específica en el frontend interceptando el `HttpResponse`, para diferenciar si el guardado falló porque el cliente fue eliminado o porque un producto desapareció.
3. **¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?** ¿Existe alguna forma en Angular de actualizar automáticamente nuestra lista desplegable (por ejemplo, con WebSockets) si otro usuario elimina un cliente, para evitar que lleguemos a tener el error 404 al intentar guardar?

4. Sobre tu nivel de comprensión de la clase de hoy, marca una opción:

- ☒ ¡Entendido! - Lo domino y podría explicarlo.
- ☐ Más o menos. - Entendí la idea general, pero tengo dudas.
- ☐ Necesito ayuda. - Me siento perdido/a con este tema.

5. ¿Cómo puedo ayudarte a comprender mejor el tema? Con más ejemplos prácticos sobre interceptores globales de errores HTTP y cómo crear validadores asíncronos para los formularios reactivos.

6. Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías? Marca una opción:

- ☒ Muy Comprometido/a: Me esforcé al máximo.
- ☐ Comprometido/a: Sé que podría haberme esforzado un poco más.
- ☐ Poco Comprometido/a: Hoy no di mi mejor esfuerzo.

7. Mi satisfacción con la clase fue... (califica del 1 al 10, donde 1 es insatisfecho y 10 es muy satisfecho).

[10]